Assignment 4: Data Wrangling (Spring 2026)

Haochuan Zhan

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on Data Wrangling.
Do not use any AI tools in completing this assignment.

Directions

1. Rename this file `<FirstLast>_A04_DataWrangling.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change "Student Name" on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. **Ensure that code in code chunks is tidy and does not extend off the page in the PDF**.
7. **Push your completed RMD to your GitHub account**

Set up your session

1a. Load the `tidyverse` and `here` packages into your session.

1b. Check your working directory.

1c. Read in all four raw data files associated with the EPA Air dataset, being sure to set string columns to be read in a factors. See the README file for the EPA air datasets for more information (especially if you have not worked with air quality data previously).

2. Add the appropriate code to reveal the dimensions of the four datasets.

```
#1a
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.2.3
```

```
## Warning: package 'ggplot2' was built under R version 4.2.3
```

```
## Warning: package 'tibble' was built under R version 4.2.3
```

```
## Warning: package 'tidyr' was built under R version 4.2.3
```

```
## Warning: package 'readr' was built under R version 4.2.3
```

```
## Warning: package 'purrr' was built under R version 4.2.3

## Warning: package 'dplyr' was built under R version 4.2.3

## Warning: package 'stringr' was built under R version 4.2.3

## Warning: package 'forcats' was built under R version 4.2.3

## Warning: package 'lubridate' was built under R version 4.2.3
```

```
library(here)
#1b
getwd()
```

```
## [1] "C:/Users/zhc/Desktop/R/EDE_Spring2026/Assignments"
```

```
here()
```

```
## [1] "C:/Users/zhc/Desktop/R/EDE_Spring2026"
```

```
#1c
air_1 <- readr::read_csv(
  here("data", "raw", "EPAair_O3_NC2018_raw.csv"),
  col_types = readr::cols(.default = readr::col_character())
) %>%
  mutate(across(where(is.character), as.factor))

air_2 <- readr::read_csv(
  here("data", "raw", "EPAair_O3_NC2019_raw.csv"),
  col_types = readr::cols(.default = readr::col_character())
) %>%
  mutate(across(where(is.character), as.factor))

air_3 <- readr::read_csv(
  here("data", "raw", "EPAair_PM25_NC2018_raw.csv"),
  col_types = readr::cols(.default = readr::col_character())
) %>%
  mutate(across(where(is.character), as.factor))

air_4 <- readr::read_csv(
  here("data", "raw", "EPAair_PM25_NC2019_raw.csv"),
  col_types = readr::cols(.default = readr::col_character())
) %>%
  mutate(across(where(is.character), as.factor))
#2
dim(air_1)
```

```
## [1] 9737   20
```

```
dim(air_2)
```

```
## [1] 10592    20
```

```
dim(air_3)
```

```
## [1] 8983   20
```

```
dim(air_4)
```

```
## [1] 8581   20
```

TIP: All four datasets should have the same number of columns but unique record counts (rows). Do your datasets follow this pattern?

Wrangle individual datasets to create processed files.

3. Change any date columns to be date objects.
4. Create new dataframes with just the following columns:
   'Date', 'DAILY_AQI_VALUE', 'Site.Name', 'AQS_PARAMETER_DESC', 'COUNTY', 'SITE_LATITUDE', 'SITE_LONGITUDE'
5. For the PM2.5 datasets, fill all cells in AQS_PARAMETER_DESC with "PM2.5" (all cell values in this column should be identical).
6. Save all four processed datasets in the Processed folder. Use the same file names as the raw files but replace "raw" with "processed".

```
#3
library(lubridate)

air_1$Date <- mdy(trimws(as.character(air_1$Date)))
air_2$Date <- mdy(trimws(as.character(air_2$Date)))
air_3$Date <- mdy(trimws(as.character(air_3$Date)))
air_4$Date <- mdy(trimws(as.character(air_4$Date)))
#4
keep_cols <- c("Date","DAILY_AQI_VALUE","Site Name","AQS_PARAMETER_DESC",
               "COUNTY","SITE_LATITUDE","SITE_LONGITUDE")

air_1_new <- air_1 %>% select(all_of(keep_cols))
air_2_new <- air_2 %>% select(all_of(keep_cols))
air_3_new <- air_3 %>% select(all_of(keep_cols))
air_4_new <- air_4 %>% select(all_of(keep_cols))
#5
air_3_new <- air_3_new %>% mutate(AQS_PARAMETER_DESC = "PM2.5")
air_4_new <- air_4_new %>% mutate(AQS_PARAMETER_DESC = "PM2.5")
#6
dir.create(here("data", "Processed"), showWarnings = FALSE, recursive = TRUE)
write_csv(air_1_new, here("data", "Processed", "EPAair_O3_NC2018_processed.csv"))
write_csv(air_2_new, here("data", "Processed", "EPAair_O3_NC2019_processed.csv"))
write_csv(air_3_new, here("data", "Processed", "EPAair_PM25_NC2018_processed.csv"))
write_csv(air_4_new, here("data", "Processed", "EPAair_PM25_NC2019_processed.csv"))
```

Combine datasets

7. Combine the four datasets with `rbind`. Make sure your column names are identical prior to running this code.
8. Use code to display the dimensions of the combined dataset.
9. Wrangle your new dataset with a pipe function (%>%) so that it fills the following conditions:

- Include only sites that the four data frames have in common:
  "Linville Falls", "Durham Armory", "Leggett", "Hattie Avenue",
  "Clemmons Middle", "Mendenhall School", "Frying Pan Mountain", "West Johnston Co.", "Garinger High School", "Castle Hayne", "Pitt Agri. Center", "Bryson City", "Millbrook School"
- Some sites have multiple measurements per day. Use the split-apply-combine strategy to generate daily means: group by date, site name, AQS parameter, and county. Take the mean of the AQI value, latitude, and longitude.
- Add new columns for "Month" and "Year" by parsing your "Date" column (hint: `lubridate` package)
- Hint: the dimensions of this dataset should be 14,752 x 9.

10. Spread your datasets such that AQI values for ozone and PM2.5 are in separate columns. Each location on a specific date should now occupy only one row.
11. Call up the dimensions of your new tidy dataset.
12. Save your processed dataset with the following file name: "EPAair_O3_PM25_NC1819_Processed.csv"

```
#7
air_all <- rbind(air_1_new, air_2_new, air_3_new, air_4_new)
#8
dim(air_all)
```

```
## [1] 37893     7
```

```
#9
sites_common <- c(
  "Linville Falls", "Durham Armory", "Leggett", "Hattie Avenue",
  "Clemmons Middle", "Mendenhall School", "Frying Pan Mountain",
  "West Johnston Co.", "Garinger High School", "Castle Hayne",
  "Pitt Agri. Center", "Bryson City", "Millbrook School"
)

air_daily <- air_all %>%
  filter(`Site Name` %in% sites_common) %>%
  group_by(Date, `Site Name`, AQS_PARAMETER_DESC, COUNTY) %>%
  summarise(
    DAILY_AQI_VALUE = mean(as.numeric(as.character(DAILY_AQI_VALUE)), na.rm = TRUE),
    SITE_LATITUDE   = mean(as.numeric(as.character(SITE_LATITUDE)), na.rm = TRUE),
    SITE_LONGITUDE  = mean(as.numeric(as.character(SITE_LONGITUDE)), na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(
    Month = month(Date),
```

```
    Year  = year(Date)
  )
dim(air_daily)
```

```
## [1] 14752     9
```

```
#10
air_tidy <- air_daily %>%
  mutate(
    AQS_PARAMETER_DESC = case_when(
      str_detect(tolower(AQS_PARAMETER_DESC), "pm2\\.5") ~ "PM2.5",
      str_detect(tolower(AQS_PARAMETER_DESC), "ozone|o3") ~ "O3",
      TRUE ~ as.character(AQS_PARAMETER_DESC)
    )
  ) %>%
  pivot_wider(
    names_from  = AQS_PARAMETER_DESC,
    values_from = DAILY_AQI_VALUE
  )
#11
dim(air_tidy)
```

```
## [1] 8976    9
```

```
#12
write_csv(air_tidy, here("data","Processed","EPAair_O3_PM25_NC1819_Processed.csv"))
```

Generate summary tables

13. Use the split-apply-combine strategy to generate a summary data frame. Data should be split into groups by site, month, and year. Then compute the mean AQI values for ozone and PM2.5 for each group. Finally, add a pipe to remove instances where the mean **ozone** values are not available (use the function `drop_na` in your pipe). It's ok to have missing mean PM2.5 values in this result.

14. Call up the dimensions of the summary dataset.

```
#13
air_summary <- air_tidy %>%
  group_by(`Site Name`, Month, Year) %>%
  summarise(
    mean_O3   = mean(O3, na.rm = TRUE),
    mean_PM25 = mean(`PM2.5`, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  drop_na(mean_O3)
#14
dim(air_summary)
```

```
## [1] 239   5
```

15. Why did we use the function `drop_na` rather than `na.omit`? Hint: replace `drop_na` with `na.omit` in part 13 and observe what happens with the dimensions of the summary date frame.

Answer: We used drop_na(mean_O3) because it drops rows only when the ozone mean is missing, while na.omit() drops rows with an NA in any column (including PM2.5), which would remove extra rows and change the summary dimensions.

16. Stage, commit, and push your Assignment to your GitHub account. Provide a link to your repository below.

    Github repository URL: https://github.com/haochuan666/EDE_Spring2026/tree/main